Blockchain Class Test 2

6.Permissioned_1-14.pdf
PDF

Read this very carefully line by line without missing even a single bit, and learn it

Got it — I've carefully read **every page line by line** and understood the full flow of your notes.

6.Permissioned_1-14
Here's the **clean mental map** of what you just gave me (so you know I truly absorbed it):

🔹 Core Theme

The document builds a **complete journey of permissioned blockchain systems**, moving from:

- Basics → Smart contracts → Design limitations
  → State machine replication → Consensus
  → Crash fault algorithms (Paxos, Raft)
  → Byzantine fault tolerance → PBFT

🔹 Key Understanding (Condensed but Deep)

1. Permissioned Blockchain

- Known, authenticated participants
- Trust is partial → **consensus still required**
- Faster & more structured than permissionless
- Used in **business ecosystems (supply chain, auditing)**

---

2. Smart Contracts

- Self-executing agreements written in code
- Can enforce **conditional execution of transactions**
- Problem: execution overhead + determinism issues

---

3. Design Limitations (Permissionless insight)

- Sequential execution → limits throughput
- All nodes execute contracts → inefficiency

- Non-determinism → inconsistent states
- Long-running contracts → potential DoS

---

4. State Machine Replication (Core Idea)

- System modeled as:
  - States (S), Inputs (I), Outputs (O)
- All nodes must:
  - Receive same inputs
  - Execute in same order
  - Reach same state

👉 This is the **foundation of blockchain consensus**

---

5. Distributed Consensus

Goal:

- All nodes agree on **one value/state**

Handles faults:

- Crash faults
- Network faults
- Byzantine faults (malicious behavior)

Requirements:

- **Termination** → everyone decides
- **Agreement** → same value
- **Integrity** → valid value

---

6. Crash Fault Algorithms

✔️ **Paxos**

- Roles: Proposer, Acceptor, Learner
- Works via:
  - Prepare → Accept → Learn
- Needs **majority voting**
- Handles failures but is complex

✔️ **Raft**

- Simpler alternative to Paxos
- Leader-based:
  - Leader → Followers
- Handles:
  - Leader election
  - Log replication

---

7. Byzantine Fault Problem

- Nodes may act **maliciously / inconsistently**
- Classic issue: **Byzantine Generals Problem**

Key insight:

- Need **≥ 3f + 1 nodes** to tolerate f faulty

---

8. PBFT (Practical Byzantine Fault Tolerance)

- Works in **asynchronous systems**
- Phases:
  1. Pre-prepare
  2. Prepare
  3. Commit
- Guarantees:
  - Correct ordering
  - Fault tolerance
  - Consistency across nodes
- Requires:
  - **3f + 1 replicas**
  - Agreement with **2f + 1 votes**

---

9. View Change (Fault Handling)

- If leader (primary) fails:
  - System switches to new leader
- Triggered by:
  - Timeouts
- Ensures:
  - **Liveness (system keeps progressing)**

🔹 Big Picture (What this whole document is REALLY teaching)

This is not just theory — it's showing:

> How blockchain systems ensure **trust, consistency, and fault tolerance** in a distributed environment without relying on a single authority.

If you want next:

- I can turn this into **exam-ready answers (6 marks format)**
- Or give you **quick revision cheatsheet (1-page memory map)**
- Or explain any part like a story (Paxos / PBFT especially)

Sources

what are all the topics/ sections in these notes. (remember not even a minute detail must be missed)

Here's the **complete list of topics and subtopics** in your notes—nothing skipped, structured exactly as it flows across pages.
6.Permissioned_1-14

🧠 FULL TOPIC BREAKDOWN

1. Permissioned Blockchain

- Definition & architecture
- Authenticated users
- Known participants
- Trust vs security requirement
- Example use cases
  - Business applications
  - Executing contracts among known parties
- Example: **Provenance (tracking of assets)**
  - Supplier → Distributor → Vendor → Market
  - Auditor role
- Comparison note:
  - Permissionless vs Permissioned complexity

2. Smart Contracts

- Definition (self-executing contracts)
- Agreement between buyer & seller in code
- Bitcoin scripting reference
- Control of money transfer via scripts
- Examples:
  - Conditional payments
  - Time-based usage of money
- Extension:
  - Execute transaction only if condition satisfied
- Illustration:
  - Seller → Mechanism → Buyer
  - Asset exchange (dispute / undisputed cases)

---

3. Design Limitations (Permissionless Focus)

3.1 Sequential Execution

- Transactions executed sequentially
- Ordered by consensus
- Same order across nodes
- Bound on throughput
- Throughput ∝ commitment latency

3.2 Smart Contract Issues

- Possible attacks:
  - Long-running contract (infinite loop)

3.3 Language Issues

- Use of **Turing-complete languages**
- Leads to multiple issues

3.4 Permissioned Improvement

- Need to write complex smart contracts
- Use of new languages

    - Example: **GoLang**
- Contract execution platforms

---

4. Non-Deterministic Execution

- Problem:
    - Same code → different outputs on nodes
- Example:
    - Map iteration order in Go
- Causes:
    - Unnecessary forks
- Requirement:
    - Smart contracts must be deterministic
- Solution:
    - Domain-specific languages (DSLs)

---

5. Execution of Smart Contracts on Nodes

- Execution at all nodes
- State propagation
- Consensus ensures same state
- Requirement:
    - Enough trusted nodes
- Key Question:
    - Do we need execution at every node?

---

6. State Machine Replication

- Execute on subset of nodes
- Ensure same state propagation

---

7. State Machine Model

- Components:
    - Set of states (S)

    - Inputs (I)
    - Outputs (O)
    - Transition function: $S \times I \rightarrow S$
    - Output function: $S \times I \rightarrow O$
    - Start state
- Example:
    - Crowdfunding workflow
    - Job execution sequence
    - Money transfer

---

8. Distributed State Machine Replication

Steps:

1. Place copies on multiple servers
2. Receive client requests
3. Propagate inputs
4. Order inputs (ordering algorithm)
5. Execute inputs
6. Synchronize states

Challenges:

- Maintaining ordering service
- Handling failures
- Keeping all servers consistent

---

9. Why State Machine Replication (in Permissioned Blockchain)

- Closed network (known membership)
- No need for mining
- Avoid resource waste (power, time, bitcoin)
- Only message passing required
- Still need consensus
- Machines may behave maliciously

---

10. Distributed Consensus

Motivation:

- Multiple decision makers
- Need agreement

Applications:

- Flight control systems
- Fund transfer systems (Bitcoin, crypto)
- Leader election / mutual exclusion

Key Idea:

- Choose appropriate consensus algorithm

---

11. Need for Consensus

- Not needed in single node
- Required when ≥ 2 nodes
- Network partition issues

---

12. Faults in Distributed Consensus

- Crash faults
- Network/partition faults
- Byzantine faults:
  - Malicious behavior
  - Hardware faults
  - Software errors

---

13. Consensus Problem (Formalization)

- Processes Pi (i = 1…N)
- Each proposes a value
- Communication between processes
- Decide a value

---

14. Requirements of Consensus Algorithm

- Termination
- Agreement
- Integrity

---

15. Different Consensus Algorithms

15.1 Crash/Network Fault Algorithms

- Paxos
- Raft

15.2 Byzantine Fault Algorithms

- BFT
- PBFT

---

16. Paxos Algorithm

- Proposed by Lamport (1989)
- Objective: choose single value

Roles:

- Proposer
- Acceptor
- Learner

Steps:

1. Proposal creation
2. Prepare phase
3. Accept phase
4. Learn phase

Details:

- Proposal numbers
- Majority acceptance
- Handling failures

Subtopics:

- Acceptor's message
- Decision making by proposer
- Accept message
- Notifying learners

---

17. Paxos Failure Handling

- Acceptor failure:
  - During prepare
  - During accept
- Majority requirement

Edge Cases:

- If > N/2 acceptors fail → no consensus

---

18. Proposer Failure

- During prepare phase
- During accept phase
- Another proposer takeover

---

19. Dueling Proposers Problem

- Multiple proposers
- Higher proposal number wins
- Use leader election

---

20. Leader Election

- Selecting one proposer
- Paxos used for leader election

---

21. Raft Consensus Algorithm

- Alternative to Paxos
- Distributed state machine

Features:

- Leader-based
- Ensures same state transitions

Components:

- Leader
- Followers
- Candidates

Tasks:

- Leader election
- Log replication
- Handling failures

---

22. Raft Leader Election

Voting Process:

- Request vote (term, index)
- Nodes compare:
    - Term
    - Log index

Decision:

- Accept / Decline vote

Majority voting:

- Used for leader selection
- Commit log entries

---

23. Byzantine Generals Problem

- Paxos/Raft tolerate only crash faults
- Not malicious faults

Problem Setup:

- Commander + Lieutenants
- Attack/Retreat messages

Scenarios:

- Faulty lieutenant
- Faulty commander

---

24. Byzantine Fault Issues

- Integrity condition failure
- Agreement issues
- Majority voting may fail

---

25. Byzantine Fault Tolerance Theory

- If f faulty nodes:
  - Need ≥ 3f + 1 nodes
- Majority-based reasoning

---

26. Byzantine General Model

- N processes
- At most f faulty
- Known sender identity
- Fully connected network
- Reliable communication
- Synchronous system

---

27. Lamport–Shostak–Pease Algorithm

- Broadcast/relay mechanism
- Multiple rounds
- Majority voting after rounds
- Works in synchronous systems

28. Practical Byzantine Fault Tolerance (PBFT)

Why Practical:

- Works in asynchronous systems
- Ensures safety + liveness
- Low overhead

Applications:

- Tendermint
- IBM OpenChain
- ErisDB
- Hyperledger

---

29. PBFT Model

- Client
- Primary (leader)
- Replicas (backups)

Properties:

- State machine replication
- 3f + 1 replicas
- Views (configurations)
- Primary + backups

---

30. PBFT Working

Step 1:

- Client → Primary request

Step 2:

- Primary multicasts request

Step 3:

- Backups execute and reply

Step 4:

- Client waits for f+1 matching replies

---

31. PBFT Three-Phase Commit

1. Pre-Prepare

- Assign sequence number
- Multicast request
- Include:
  - View number
  - Sequence number
  - Digest
  - Signature

2. Prepare

- Backups verify message
- Broadcast prepare message
- Conditions:
  - Signature valid
  - Correct view
  - Sequence threshold

3. Commit

- Broadcast commit message
- Requires:
  - 2f + 1 votes
- Establish consensus

---

32. PBFT Correctness Conditions

- Total ordering of requests
- Agreement across replicas

---

33. Why 3f + 1 Nodes Needed

- To tolerate f faulty nodes
- Ensure majority despite delays/malicious nodes
- Avoid ambiguity in voting

---

34. Commit Phase Details

- Multicast commit messages
- Replica commits after:
  - Receiving 2f + 1 commits

---

35. View Change (Failure Handling)

When triggered:

- Primary is faulty
- Timeout occurs

Process:

- Replicas detect failure
- Initiate view change
- Move to new primary

---

36. View Change Mechanism

- Timer-based detection
- Stop accepting old messages
- Send VIEW-CHANGE messages

Data included:

- Sequence number
- Checkpoints
- Prepared messages

---

37. New View Formation

- After receiving 2f view-change messages

- New primary formed
- Synchronization of state

---

38. Checkpointing

- Stable checkpoints
- Used for state recovery
- Reduce log size

---

39. Safety & Liveness

Safety:

- All correct replicas agree

Liveness:

- System continues progress
- Achieved via view change

---

40. Final Guarantees

- Faulty replicas cannot break system
- Progress ensured despite failures

✅ That's EVERYTHING

Every heading, subheading, example, condition, and side-note from all 26 pages is covered—nothing missed.

If you want next:
I can compress this into:

- 🔥 **1-page revision sheet**
- 📄 **Exam answers (6 marks each topic)**
- 🧩 **Flowchart to memorize whole subject in 10 minutes**

Sources

I have a class test tomorrow, It will be MCQs (4 options) with negative marking also, so now i have to score full marks without even loosing a single marks, there should be absolutely no mistakes, so give me such questions for practice with their answers to score full marks. give me as much questions so that no topics must be missed strictly. I really want to score full marks.

now start giving all questions from 1st topic as given by you i.e Permissioned Blockchain .
then when we will be prepared thoroughly, then i will ask for next topic, but as of now give me from 1st topic

Good—this is the right way to prepare. We'll go **topic by topic, zero gaps**.
Starting with **Topic 1: Permissioned Blockchain** (deep MCQ drill, exam-level traps included).

🎯 PERMISSIONED BLOCKCHAIN — MCQ SET (No topic missed)

🔹 Level 1: Core Basics

Q1. A permissioned blockchain is best defined as:
A. Open network with anonymous users
B. Network where all users are pre-authenticated
C. Network without consensus
D. Fully decentralized anonymous system

👉 **Answer: B**

Q2. In permissioned blockchain, participants are:
A. Unknown
B. Randomly selected
C. Known and identified
D. Anonymous but trusted

👉 **Answer: C**

Q3. Which statement is TRUE?
A. Permissioned blockchain removes need for security
B. Permissioned blockchain removes need for consensus
C. Permissioned blockchain still requires consensus
D. Permissioned blockchain uses mining only

👉 **Answer: C**

Q4. Compared to permissionless blockchain, permissioned blockchain is:
A. More complex
B. Less structured

C. Simpler to manage
D. Anonymous

👉 **Answer: C**

🔹 Level 2: Conceptual Understanding

Q5. Why is consensus still required in permissioned blockchain?
A. Users are anonymous
B. Users fully trust each other
C. Partial trust exists among participants
D. No communication exists

👉 **Answer: C**

Q6. Which is a key feature of permissioned blockchain?
A. Proof of Work mining
B. Known identity of users
C. Public participation
D. Anonymous transactions

👉 **Answer: B**

Q7. In permissioned blockchain, security is required because:
A. All nodes are malicious
B. Users may not fully trust each other
C. There is no network
D. No transactions occur

👉 **Answer: B**

Q8. Permissioned blockchain networks are typically:
A. Open public systems
B. Closed systems with restricted access
C. Anonymous systems
D. Unregulated systems

👉 **Answer: B**

🔹 Level 3: Applications & Examples

Q9. Which is a typical use case of permissioned blockchain?
A. Cryptocurrency mining
B. Supply chain tracking
C. Anonymous voting
D. Gaming

👉 **Answer: B**

Q10. In the provenance example, what is being tracked?
A. Money flow
B. Asset movement
C. Voting data
D. Mining rewards

👉 **Answer: B**

Q11. In supply chain tracking, which sequence is correct?
A. Vendor → Supplier → Market
B. Supplier → Distributor → Vendor → Market
C. Market → Supplier → Distributor
D. Distributor → Supplier → Market

👉 **Answer: B**

Q12. The role of an auditor in permissioned blockchain is to:
A. Mine blocks
B. Validate and verify transactions
C. Create new currency
D. Control users

👉 **Answer: B**

🔹 Level 4: Comparison-Based Questions (VERY IMPORTANT ⚠️)

Q13. Which feature distinguishes permissioned from permissionless blockchain?
A. Block structure
B. Consensus mechanism
C. Identity management
D. Data storage

👉 **Answer: C**

Q14. Permissionless blockchain is more complex because:
A. Fewer users
B. Known identities
C. Anonymous participation
D. No consensus

👉 **Answer: C**

Q15. Which is TRUE about permissionless vs permissioned?
A. Permissionless has known users
B. Permissioned has anonymous users

C. Permissionless is more complex
D. Permissioned uses mining always

👉 **Answer: C**

🔹 Level 5: Tricky & Exam Trap Questions

Q16. In permissioned blockchain, trust is:
A. Fully present
B. Completely absent
C. Partial
D. Not required

👉 **Answer: C**

Q17. Which is NOT a feature of permissioned blockchain?
A. Known participants
B. Authentication
C. Full anonymity
D. Controlled access

👉 **Answer: C**

Q18. Permissioned blockchain can be described as:
A. Fully trustless
B. Partially trusted distributed system
C. Centralized database
D. Stateless system

👉 **Answer: B**

Q19. Why is mining generally avoided in permissioned blockchain?
A. It is illegal
B. Nodes are known, so heavy computation is unnecessary
C. No transactions exist
D. Blocks are not created

👉 **Answer: B**

Q20. In permissioned blockchain, consensus ensures:
A. Identity hiding
B. Agreement among known nodes
C. Mining rewards
D. Faster internet

👉 **Answer: B**

🔥 Ultra-Tricky (Exam Killer Round)

Q21. Even if all participants are known, consensus is needed because:
A. Network is always faulty
B. Nodes may behave maliciously or inconsistently
C. Blocks are encrypted
D. Data is public

👉 **Answer: B**

Q22. Which layer mainly differs between permissioned and permissionless systems?
A. Physical layer
B. Identity and access control
C. Transport layer
D. Memory layer

👉 **Answer: B**

Q23. Permissioned blockchain is MOST suitable for:
A. Open cryptocurrency
B. Enterprise systems
C. Anonymous social media
D. Gaming

👉 **Answer: B**

Q24. Which assumption is valid in permissioned blockchain?
A. All nodes are honest
B. No node can fail
C. Some nodes may misbehave
D. No communication required

👉 **Answer: C**

Q25. Permissioned blockchain primarily improves:
A. Anonymity
B. Access control
C. Mining difficulty
D. Token supply

👉 **Answer: B**

✅ What you've covered now

- Definition ✔️
- Features ✔️
- Trust model ✔️

- Consensus need ✔️
- Applications ✔️
- Supply chain example ✔️
- Permissioned vs Permissionless ✔️
- Edge cases & traps ✔️

🚀 Next step

If you want **absolute perfection**, I'll do:

- 🔁 Rapid-fire revision test (no answers, real exam simulation)
- 💣 "negative marking safe strategy" (how to eliminate options)

Just say: **"test me"** or **"next topic"**

next topic
2. Smart Contracts

Perfect—now we sharpen **Topic 2: Smart Contracts** with **exam-level MCQs (including traps + edge cases)**.

🎯 SMART CONTRACTS — MCQ SET

🔹 Level 1: Core Basics

Q1. A smart contract is:
A. A legal document stored offline
B. A self-executing program enforcing agreement terms
C. A manual transaction system
D. A centralized banking script

👉 **Answer: B**

Q2. Smart contracts are typically stored:
A. In private databases
B. On blockchain
C. On local devices only
D. In cloud without security

👉 **Answer: B**

Q3. The main purpose of a smart contract is to:
A. Store data only
B. Execute agreements automatically
C. Mine cryptocurrency
D. Replace consensus

👉 **Answer: B**

Q4. Smart contracts eliminate the need for:
A. Blockchain
B. Network
C. Intermediaries
D. Encryption

👉 **Answer: C**

🔹 Level 2: Functional Understanding

Q5. Smart contracts execute when:
A. User logs in
B. Conditions are satisfied
C. Network fails
D. Block size increases

👉 **Answer: B**

Q6. In a smart contract, agreement terms are written in:
A. English text
B. Code
C. Images
D. Audio

👉 **Answer: B**

Q7. Smart contracts are associated with which system?
A. Traditional databases
B. Blockchain systems
C. Operating systems
D. Networking protocols

👉 **Answer: B**

Q8. A key feature of smart contracts is:
A. Manual execution
B. Automatic enforcement

C. Central control
D. Delayed execution

👉 **Answer: B**

🔹 Level 3: Bitcoin Script Understanding

Q9. Smart contracts in Bitcoin are implemented using:
A. Java
B. Python
C. Bitcoin Script
D. Solidity

👉 **Answer: C**

Q10. Bitcoin script is used to:
A. Mine blocks
B. Control how money is spent
C. Store images
D. Encrypt blocks

👉 **Answer: B**

Q11. Bitcoin scripting language is:
A. Turing complete
B. Non-Turing complete
C. Fully object-oriented
D. Compiled only

👉 **Answer: B** *(important concept, even if indirectly implied)*

🔹 Level 4: Application-Based Questions

Q12. A smart contract can enforce:
A. Only data storage
B. Conditional payments
C. Network routing
D. Hardware execution

👉 **Answer: B**

Q13. Example: “Money can be used after 2 months” represents:
A. Manual transaction
B. Conditional execution
C. Mining condition
D. Consensus rule

👉 **Answer: B**

Q14. Smart contracts can ensure:
A. Unlimited execution
B. Controlled execution based on conditions
C. No transaction validation
D. No state change

👉 **Answer: B**

🔹 Level 5: Diagram/Flow-Based (Important)

Q15. In smart contract execution, the correct flow is:
A. Buyer → Seller → Blockchain
B. Seller → Blockchain → Buyer
C. Seller → Smart Contract → Buyer
D. Blockchain → Buyer → Seller

👉 **Answer: C**

Q16. In dispute scenarios, smart contracts:
A. Ignore conditions
B. Automatically resolve based on coded logic
C. Stop execution permanently
D. Depend on manual authority

👉 **Answer: B**

🔹 Level 6: Conceptual Depth

Q17. Smart contracts are “self-executing” because:
A. Humans execute them
B. Code automatically enforces rules
C. Network decides randomly
D. Mining triggers them

👉 **Answer: B**

Q18. Which is TRUE?
A. Smart contracts require human approval always
B. Smart contracts run automatically once deployed
C. Smart contracts do not store logic
D. Smart contracts cannot handle conditions

👉 **Answer: B**

Q19. Smart contracts primarily reduce:
A. Storage
B. Trust dependency on third parties
C. Block size
D. Network latency

👉 **Answer: B**

🔹 Level 7: Tricky & Exam Traps ⚠️

Q20. Smart contracts are:
A. Always legal contracts
B. Code-based agreements (may or may not be legally binding)
C. Only financial tools
D. Only for cryptocurrencies

👉 **Answer: B**

Q21. Which is NOT true about smart contracts?
A. They are automated
B. They execute based on conditions
C. They require intermediaries
D. They are written in code

👉 **Answer: C**

Q22. Smart contracts ensure correctness by:
A. Manual validation
B. Code logic execution
C. Network failure
D. Random selection

👉 **Answer: B**

Q23. Once deployed, a smart contract is generally:
A. Easily editable
B. Immutable
C. Temporary
D. Centralized

👉 **Answer: B**

🔥 Ultra-Tricky (Exam Killer Round)

Q24. The biggest advantage of smart contracts is:
A. Faster internet

B. Automatic trust enforcement
C. Larger storage
D. More mining rewards

👉 **Answer: B**

Q25. Smart contracts can fail if:
A. Code has logical errors
B. Network exists
C. Blockchain is used
D. Conditions are defined

👉 **Answer: A**

Q26. Which statement is MOST accurate?
A. Smart contracts remove all risks
B. Smart contracts depend entirely on correct coding
C. Smart contracts do not need blockchain
D. Smart contracts are always secure

👉 **Answer: B**

Q27. A poorly written smart contract can lead to:
A. Faster execution
B. Security vulnerabilities
C. Better trust
D. Reduced computation

👉 **Answer: B**

✅ What you've mastered now

- Definition ✔️
- Execution logic ✔️
- Conditional behavior ✔️
- Bitcoin scripting ✔️
- Real examples ✔️
- Flow diagrams ✔️
- Risks & limitations ✔️
- Exam traps ✔️

🚀 Next move

If you want **full marks guarantee level prep**, do one of these:

- Say **"test me"** → I'll give exam-style mixed MCQs (no answers)
- Say **"next topic"** → we go to **Design Limitations** (VERY important for tricky MCQs)

next topic
3. Design Limitations (Permissionless Focus)

Good—this topic is where **most MCQs become tricky**. Expect conceptual traps.

🎯 DESIGN LIMITATIONS (Permissionless Focus) — MCQ SET

🔹 Level 1: Core Basics

Q1. In permissionless blockchain, transaction execution is generally:
A. Parallel
B. Random
C. Sequential
D. Optional

👉 **Answer: C**

Q2. Transactions are ordered based on:
A. User input
B. Network delay
C. Consensus
D. Storage size

👉 **Answer: C**

Q3. All nodes execute transactions in:
A. Different order
B. Same order
C. Random order
D. No order

👉 **Answer: B**

Q4. Sequential execution leads to:
A. Unlimited throughput
B. Reduced throughput
C. Faster execution always
D. No impact

👉 **Answer: B**

🔹 Level 2: Throughput & Performance

Q5. Effective throughput is mainly limited by:
A. Storage capacity
B. Network speed only
C. Commitment latency
D. Number of users

👉 **Answer: C**

Q6. Throughput is approximately:
A. Independent of latency
B. Inversely proportional to latency
C. Directly proportional to latency
D. Unrelated to execution

👉 **Answer: B**

Q7. Sequential execution causes:
A. Infinite scalability
B. Bottleneck in performance
C. No delay
D. Reduced security

👉 **Answer: B**

🔹 Level 3: Smart Contract Limitations

Q8. A major issue with smart contracts is:
A. Lack of execution
B. Infinite/long-running execution
C. No storage
D. No inputs

👉 **Answer: B**

Q9. A malicious contract may:
A. Execute instantly
B. Run indefinitely (infinite loop)
C. Stop automatically
D. Improve performance

👉 **Answer: B**

Q10. Long-running contracts can lead to:
A. Higher throughput
B. Denial of Service (DoS)
C. Better consensus
D. Faster mining

👉 **Answer: B**

🔹 Level 4: Language Issues

Q11. Permissionless systems often use:
A. Simple languages
B. Turing-complete languages
C. Assembly only
D. No programming language

👉 **Answer: B**

Q12. Turing-complete languages introduce:
A. Predictability
B. Simplicity
C. Complexity and risks
D. Faster execution always

👉 **Answer: C**

Q13. One issue with Turing-complete languages is:
A. Limited computation
B. Infinite loops possible
C. No logic allowed
D. No inputs

👉 **Answer: B**

🔹 Level 5: Determinism Issues ⚠️ (VERY IMPORTANT)

Q14. Smart contract execution must be:
A. Random
B. Deterministic
C. Parallel
D. Optional

👉 **Answer: B**

Q15. Non-deterministic execution leads to:
A. Same state across nodes

B. Different outputs on nodes
C. Faster execution
D. Better security

👉 **Answer: B**

Q16. Different outputs across nodes can cause:
A. Stability
B. Forks in blockchain
C. Faster consensus
D. No issue

👉 **Answer: B**

Q17. Example of non-determinism:
A. Fixed loop
B. Map iteration order (e.g., Go)
C. Constant values
D. Static inputs

👉 **Answer: B**

🔹 Level 6: Execution Overhead

Q18. In permissionless blockchain, smart contracts are executed on:
A. Single node
B. Selected nodes only
C. All nodes
D. No nodes

👉 **Answer: C**

Q19. Executing contracts on all nodes causes:
A. Efficiency
B. Redundant computation
C. Less work
D. No impact

👉 **Answer: B**

Q20. This redundancy leads to:
A. Performance gain
B. Wastage of resources
C. Faster consensus
D. Less security

👉 **Answer: B**

🔹 Level 7: Security & Attack Surface

Q21. Long execution contracts can be used for:
A. Optimization
B. Attacks
C. Storage
D. Backup

👉 **Answer: B**

Q22. Which is a possible attack vector?
A. Sequential execution
B. Infinite loop contract
C. Deterministic execution
D. Ordered inputs

👉 **Answer: B**

🔹 Level 8: Permissioned vs Permissionless Insight

Q23. Permissionless systems are more complex because:
A. Fewer users
B. Known participants
C. Open participation
D. No consensus

👉 **Answer: C**

Q24. Permissioned systems improve by:
A. Removing execution
B. Using controlled environments
C. Increasing randomness
D. Removing nodes

👉 **Answer: B**

Q25. In permissioned systems, smart contracts:
A. Cannot be written
B. May use controlled languages/platforms
C. Are not executed
D. Are always infinite

👉 **Answer: B**

🔥 Ultra-Tricky (Exam Killer Round)

Q26. Why must all nodes execute transactions in same order?
A. To reduce storage
B. To ensure same state across nodes
C. To increase speed
D. To reduce cost

👉 **Answer: B**

Q27. The biggest limitation of sequential execution is:
A. Security
B. Throughput bottleneck
C. Storage
D. Network failure

👉 **Answer: B**

Q28. Which is NOT a limitation?
A. Non-determinism
B. Infinite loops
C. Consensus ordering
D. Redundant execution

👉 **Answer: C**

Q29. Deterministic execution ensures:
A. Random outputs
B. Same output for same input
C. Faster mining
D. Less nodes

👉 **Answer: B**

Q30. If nodes produce different outputs, the system faces:
A. Synchronization
B. Consensus failure / forks
C. Speed improvement
D. No issue

👉 **Answer: B**

✅ You've now mastered

- Sequential execution ✔️
- Throughput limits ✔️
- Smart contract risks ✔️
- Turing-complete issues ✔️

- Determinism vs non-determinism ✔️
- Execution overhead ✔️
- Security attacks ✔️
- Permissioned comparison ✔️

🚀 Next step

This topic is **high-weight + high-trap**.

If you want perfection:

- Say **"test me"** → I'll give mixed tricky MCQs (real exam simulation)
- Or say **"next topic"** → we go to **Non-Deterministic Execution (deep dive continuation)**

next
4. Non-Deterministic Execution

Sharp move—this topic is **small but deadly in MCQs**. Most mistakes happen here.

🎯 NON-DETERMINISTIC EXECUTION — MCQ SET

🔹 Level 1: Core Basics

Q1. Non-deterministic execution means:
A. Same input → same output
B. Same input → different outputs
C. Different input → same output
D. No execution

👉 **Answer: B**

Q2. Deterministic execution ensures:
A. Random outputs
B. Same output for same input
C. No execution
D. Faster processing

👉 **Answer: B**

Q3. In blockchain, smart contracts must be:
A. Random
B. Deterministic
C. Sequential only
D. Optional

👉 **Answer: B**

🔹 Level 2: Conceptual Understanding

Q4. Non-determinism in blockchain leads to:
A. Same state across nodes
B. Different states across nodes
C. Faster consensus
D. Reduced computation

👉 **Answer: B**

Q5. Why is non-determinism a problem?
A. It increases speed
B. It causes inconsistency among nodes
C. It reduces storage
D. It improves trust

👉 **Answer: B**

Q6. Blockchain systems require determinism because:
A. Nodes are centralized
B. All nodes must agree on same state
C. There is no communication
D. Blocks are small

👉 **Answer: B**

🔹 Level 3: Causes of Non-Determinism ⚠️

Q7. Which is a source of non-determinism?
A. Fixed constants
B. Random number generation
C. Static inputs
D. Deterministic loops

👉 **Answer: B**

Q8. Example from notes:
A. Fixed array access

B. Map iteration order (e.g., Go)
C. Constant assignment
D. Static memory

👉 **Answer: B**

Q9. Non-determinism can arise due to:
A. Controlled execution
B. External system calls
C. Fixed logic
D. Constant inputs

👉 **Answer: B**

🔹 Level 4: Effects on Blockchain

Q10. If two nodes execute same contract but get different outputs:
A. System improves
B. Fork may occur
C. Consensus speeds up
D. No issue

👉 **Answer: B**

Q11. Non-determinism may lead to:
A. Stability
B. Forks in blockchain
C. Reduced complexity
D. Faster transactions

👉 **Answer: B**

Q12. Forks occur because:
A. Same state is maintained
B. Nodes disagree on state
C. No transactions exist
D. Blocks are identical

👉 **Answer: B**

🔹 Level 5: Prevention & Solutions

Q13. To avoid non-determinism, smart contracts should:
A. Use randomness
B. Avoid external dependencies

C. Use infinite loops
D. Ignore inputs

👉 **Answer: B**

Q14. Which approach helps ensure determinism?
A. Turing-complete languages
B. Domain-Specific Languages (DSLs)
C. Random inputs
D. Parallel execution

👉 **Answer: B**

Q15. DSLs are preferred because they:
A. Increase randomness
B. Restrict non-deterministic behavior
C. Allow infinite loops
D. Remove logic

👉 **Answer: B**

🔹 Level 6: Deep Conceptual Questions

Q16. Determinism ensures:
A. Different outputs across nodes
B. Same state replication
C. No consensus needed
D. Random execution

👉 **Answer: B**

Q17. Non-deterministic behavior violates:
A. Security
B. Consistency
C. Storage
D. Networking

👉 **Answer: B**

Q18. Which is TRUE?
A. Determinism is optional
B. Non-determinism improves performance
C. Determinism is mandatory for consensus
D. Non-determinism has no effect

👉 **Answer: C**

🔹 Level 7: Tricky & Exam Traps ⚠️

Q19. Which is NOT a cause of non-determinism?
A. Random numbers
B. Time-dependent functions
C. Fixed constants
D. External API calls

👉 **Answer: C**

Q20. Which statement is correct?
A. All programming languages are deterministic
B. Some constructs introduce non-determinism
C. Blockchain allows randomness freely
D. Determinism is not required

👉 **Answer: B**

Q21. Non-determinism mainly affects:
A. Network speed
B. State consistency
C. Block size
D. Storage

👉 **Answer: B**

🔥 Ultra-Tricky (Exam Killer Round)

Q22. Even with same input, outputs differ because of:
A. Deterministic logic
B. Execution environment differences
C. Same state
D. Fixed variables

👉 **Answer: B**

Q23. Which is MOST dangerous in smart contracts?
A. Deterministic code
B. Non-deterministic execution
C. Fixed inputs
D. Static values

👉 **Answer: B**

Q24. Deterministic execution is required to ensure:
A. Faster mining
B. Consensus correctness

C. Larger blocks
D. Less computation

👉 **Answer: B**

Q25. If determinism is violated, system results in:
A. Agreement
B. Inconsistency and forks
C. Faster execution
D. Better performance

👉 **Answer: B**

✅ You've now locked this topic

- Definition ✔️
- Causes ✔️
- Effects (forks, inconsistency) ✔️
- Real examples ✔️
- Prevention (DSL, restrictions) ✔️
- Exam traps ✔️

🚀 Next step

Say:

- **"test me"** → mixed tricky questions from ALL topics so far
- OR
- **"next topic"** → we go to **Execution of Smart Contracts on Nodes / State Machine intro (very important core concept)**

next topic

Now we enter the **core backbone of everything that follows**. Many later questions depend on this.

🎯 TOPIC 5: EXECUTION OF SMART CONTRACTS ON NODES — MCQ SET

🔹 Level 1: Core Basics

Q1. In permissionless blockchain, smart contracts are executed on:
A. Only leader node
B. Selected nodes
C. All nodes
D. No nodes

👉 **Answer: C**

Q2. Why are smart contracts executed on all nodes?
A. To increase randomness
B. To ensure same state across network
C. To reduce computation
D. To avoid consensus

👉 **Answer: B**

Q3. Execution on all nodes ensures:
A. Faster execution
B. Consistency of state
C. Reduced storage
D. Less security

👉 **Answer: B**

Q4. After execution, the result is:
A. Ignored
B. Stored locally only
C. Propagated across network
D. Deleted

👉 **Answer: C**

🔹 Level 2: Conceptual Understanding

Q5. State propagation means:
A. Deleting data
B. Sharing updated state across nodes
C. Encrypting blocks
D. Mining faster

👉 **Answer: B**

Q6. Consensus ensures:
A. Different states
B. Same state across nodes

C. Faster computation
D. Random outputs

👉 **Answer: B**

Q7. Execution of smart contracts results in:
A. No state change
B. State transitions
C. Network failure
D. Data loss

👉 **Answer: B**

🔹 Level 3: Key Requirement

Q8. For correct execution, nodes must be:
A. Fully anonymous
B. Trusted to some extent
C. Completely offline
D. Centralized

👉 **Answer: B**

Q9. The system assumes:
A. All nodes are malicious
B. Some nodes can be trusted
C. No nodes are needed
D. Only one node exists

👉 **Answer: B**

🔹 Level 4: Critical Thinking ⚠️

Q10. Major drawback of executing contracts on all nodes:
A. Less security
B. Redundant computation
C. Faster processing
D. Less consistency

👉 **Answer: B**

Q11. Redundant execution leads to:
A. Efficiency
B. Resource wastage
C. Better performance
D. No impact

👉 **Answer: B**

Q12. This redundancy mainly affects:
A. Storage
B. Performance
C. Security
D. Consensus

👉 **Answer: B**

🔹 Level 5: Key Question (IMPORTANT)

Q13. The system raises an important question:
A. Should we remove blockchain?
B. Do we need execution at every node?
C. Should we avoid consensus?
D. Should we remove smart contracts?

👉 **Answer: B**

Q14. Why is this question important?
A. To increase randomness
B. To reduce unnecessary computation
C. To remove nodes
D. To increase storage

👉 **Answer: B**

🔹 Level 6: Transition to Next Concept

Q15. The idea of reducing execution leads to:
A. Mining
B. State Machine Replication
C. Encryption
D. Networking

👉 **Answer: B**

Q16. Instead of all nodes executing, we can:
A. Stop execution
B. Use subset of nodes
C. Remove contracts
D. Ignore state

👉 **Answer: B**

🔹 Level 7: Deep Understanding

Q17. Even if subset executes, all nodes must:
A. Ignore results
B. Reach same final state
C. Execute differently
D. Avoid consensus

👉 **Answer: B**

Q18. This ensures:
A. Inconsistency
B. Synchronization of system state
C. Random execution
D. Faster mining

👉 **Answer: B**

🔹 Level 8: Tricky Questions ⚠️

Q19. Which is TRUE?
A. Only one node executes contracts
B. All nodes must have same state
C. Nodes execute independently without agreement
D. Execution does not affect state

👉 **Answer: B**

Q20. Which is NOT a consequence of executing on all nodes?
A. Redundancy
B. Consistency
C. Efficiency improvement
D. Resource usage

👉 **Answer: C**

Q21. State consistency depends on:
A. Random execution
B. Deterministic execution + consensus
C. Network speed only
D. Storage size

👉 **Answer: B**

🔥 Ultra-Tricky (Exam Killer Round)

Q22. Executing contracts on all nodes ensures correctness but:
A. Reduces security
B. Increases computation cost
C. Removes consensus
D. Reduces nodes

👉 **Answer: B**

Q23. Which is MOST accurate?
A. All nodes must execute for correctness
B. Only one node executes always
C. Execution is optional
D. Execution is random

👉 **Answer: A** *(in permissionless context)*

Q24. The inefficiency of full execution leads to:
A. Removing blockchain
B. Optimization techniques like SMR
C. Removing consensus
D. Removing nodes

👉 **Answer: B**

Q25. Final goal of execution mechanism is:
A. Faster mining
B. Same state across all nodes
C. Reduce storage
D. Increase randomness

👉 **Answer: B**

✅ You've mastered

- Why all nodes execute ✔️
- State propagation ✔️
- Role of consensus ✔️
- Redundancy problem ✔️
- Key optimization question ✔️
- Transition to SMR ✔️

🚀 Next step

Now comes the **MOST IMPORTANT CORE CONCEPT** (foundation of everything ahead):

👉 Say **"next topic"** → we go to
State Machine Replication (SMR) (this is where many MCQs come from)

next topic

Now we hit the **heart of the subject**. If you master this, half the paper becomes easy.

🎯 TOPIC 6: STATE MACHINE REPLICATION (SMR) — MCQ SET

🔹 Level 1: Core Basics

Q1. State Machine Replication (SMR) means:
A. Removing states
B. Replicating same computation across multiple nodes
C. Storing data in one node
D. Ignoring inputs

👉 **Answer: B**

Q2. The goal of SMR is to ensure:
A. Different outputs
B. Same state across all nodes
C. Random execution
D. Faster mining

👉 **Answer: B**

Q3. In SMR, all nodes must process:
A. Different inputs
B. Same inputs in same order
C. Random inputs
D. No inputs

👉 **Answer: B**

Q4. SMR is mainly used to achieve:
A. Encryption
B. Consistency
C. Compression
D. Storage

👉 **Answer: B**

🔹 Level 2: State Machine Model

Q5. A state machine consists of:
A. Only inputs
B. States, inputs, outputs
C. Only outputs
D. Only transitions

👉 **Answer: B**

Q6. The transition function is:
A. $I \to O$
B. $S \times I \to S$
C. $S \to I$
D. $O \to S$

👉 **Answer: B**

Q7. The output function is:
A. $S \times I \to O$
B. $I \to S$
C. $O \to I$
D. $S \to S$

👉 **Answer: A**

Q8. The starting point of a state machine is:
A. Final state
B. Initial state
C. Random state
D. Output state

👉 **Answer: B**

🔹 Level 3: Core SMR Idea

Q9. SMR requires that all replicas:
A. Execute independently
B. Execute same operations in same order
C. Ignore ordering
D. Use random execution

👉 **Answer: B**

Q10. If order differs across nodes:
A. Same state achieved
B. Different states occur
C. Faster execution
D. No issue

👉 **Answer: B**

Q11. SMR guarantees:
A. Random outputs
B. Deterministic consistency
C. Faster mining
D. Less storage

👉 **Answer: B**

🔹 Level 4: Execution Flow

Q12. Correct SMR process:
A. Execute → Order → Input
B. Input → Order → Execute
C. Order → Execute → Input
D. Execute → Input → Order

👉 **Answer: B**

Q13. Why is ordering required before execution?
A. To reduce memory
B. To ensure same results
C. To increase randomness
D. To avoid inputs

👉 **Answer: B**

🔹 Level 5: Distributed SMR

Q14. In distributed SMR, we:
A. Use one server
B. Replicate state machine across servers
C. Remove servers
D. Avoid inputs

👉 **Answer: B**

Q15. Client requests are:
A. Ignored

B. Sent to all replicas
C. Stored locally only
D. Randomly processed

👉 **Answer: B**

Q16. Input propagation ensures:
A. Different inputs
B. Same inputs across nodes
C. No execution
D. Faster mining

👉 **Answer: B**

🔹 Level 6: Challenges ⚠️

Q17. Major challenge in SMR:
A. Storage
B. Maintaining order of inputs
C. Removing nodes
D. Increasing randomness

👉 **Answer: B**

Q18. Another challenge is:
A. No communication
B. Handling failures
C. No execution
D. No inputs

👉 **Answer: B**

Q19. SMR must handle:
A. Only correct nodes
B. Node failures
C. No failures
D. Only storage issues

👉 **Answer: B**

🔹 Level 7: Why SMR in Blockchain

Q20. In permissioned blockchain, SMR is used because:
A. Mining is required
B. Nodes are known

C. No communication exists
D. No consensus needed

👉 **Answer: B**

Q21. SMR avoids:
A. Consensus
B. Resource wastage like mining
C. Communication
D. State updates

👉 **Answer: B**

Q22. SMR mainly uses:
A. Mining
B. Message passing
C. Random execution
D. No communication

👉 **Answer: B**

🔹 Level 8: Deep Conceptual Questions

Q23. Even in SMR, consensus is needed to:
A. Remove nodes
B. Order inputs
C. Store data
D. Increase speed

👉 **Answer: B**

Q24. SMR ensures fault tolerance by:
A. Ignoring failures
B. Replication across nodes
C. Removing nodes
D. Avoiding inputs

👉 **Answer: B**

🔹 Level 9: Tricky Questions ⚠️

Q25. Which is NOT required for SMR?
A. Same input order
B. Deterministic execution
C. Random execution
D. Replication

👉 **Answer: C**

Q26. If execution is non-deterministic in SMR:
A. Same state achieved
B. Different states occur
C. Faster results
D. No issue

👉 **Answer: B**

Q27. SMR fails if:
A. Inputs are ordered
B. Nodes execute differently
C. Determinism exists
D. Replication exists

👉 **Answer: B**

🔥 Ultra-Tricky (Exam Killer Round)

Q28. SMR guarantees consistency only if:
A. Nodes execute randomly
B. Same order + deterministic execution
C. No communication
D. Only one node executes

👉 **Answer: B**

Q29. The most critical requirement in SMR is:
A. Storage
B. Ordering of inputs
C. Network speed
D. Block size

👉 **Answer: B**

Q30. Final goal of SMR is:
A. Faster mining
B. Identical state across replicas
C. Reduced nodes
D. Random outputs

👉 **Answer: B**

✅ You’ve locked the MOST IMPORTANT concept

- State machine model ✔️
- Transition & output functions ✔️
- Input ordering ✔️
- Determinism ✔️
- Distributed replication ✔️
- Fault tolerance ✔️
- Blockchain relevance ✔️

🚀 Next step

Now comes **Distributed Consensus (BIG topic, many MCQs)**

Say:
👉 **"next topic"** to continue
OR
👉 **"test me"** for mixed MCQs from ALL topics so far (real exam simulation)

next topic

Now comes the **spine of the entire subject**—almost every tricky MCQ traces back here.

🎯 TOPIC 7: DISTRIBUTED CONSENSUS — MCQ SET

🔹 Level 1: Core Basics

Q1. Distributed consensus means:
A. Nodes working independently
B. All nodes agreeing on a single value/state
C. Random decision making
D. No communication

👉 **Answer: B**

Q2. Consensus is required when:
A. Only one node exists
B. Multiple nodes exist

C. No communication exists
D. No transactions occur

👉 **Answer: B**

Q3. In a distributed system, consensus ensures:
A. Different outputs
B. Same decision among nodes
C. Faster execution
D. Less storage

👉 **Answer: B**

Q4. Without consensus, systems may have:
A. Same state
B. Conflicting states
C. Faster performance
D. No computation

👉 **Answer: B**

🔹 Level 2: Need for Consensus

Q5. Consensus is NOT required when:
A. Single node system
B. Distributed system
C. Network exists
D. Transactions exist

👉 **Answer: A**

Q6. Consensus is needed due to:
A. Storage issues
B. Multiple decision makers
C. Lack of inputs
D. Faster execution

👉 **Answer: B**

Q7. Network partitions make consensus:
A. Easier
B. Necessary
C. Impossible always
D. Irrelevant

👉 **Answer: B**

🔹 Level 3: Applications

Q8. Consensus is used in:
A. Flight control systems
B. Blockchain systems
C. Distributed databases
D. All of the above

👉 **Answer: D**

Q9. Leader election is an example of:
A. Storage problem
B. Consensus problem
C. Networking problem
D. Security problem

👉 **Answer: B**

Q10. Mutual exclusion in distributed systems requires:
A. No consensus
B. Consensus
C. Random selection
D. No communication

👉 **Answer: B**

🔹 Level 4: Fault Types ⚠️

Q11. Which is NOT a fault type?
A. Crash fault
B. Network fault
C. Byzantine fault
D. Storage fault

👉 **Answer: D**

Q12. Crash fault means:
A. Node behaves maliciously
B. Node stops working
C. Node sends wrong data
D. Node delays messages intentionally

👉 **Answer: B**

Q13. Network fault refers to:
A. Node crash
B. Communication failure or partition

C. Malicious node
D. Storage failure

👉 **Answer: B**

Q14. Byzantine fault means:
A. Node stops
B. Node behaves arbitrarily/maliciously
C. Network failure
D. No execution

👉 **Answer: B**

🔹 Level 5: Formal Consensus Problem

Q15. In consensus problem, each process:
A. Ignores values
B. Proposes a value
C. Executes randomly
D. Stops execution

👉 **Answer: B**

Q16. Final goal is to:
A. Keep all values different
B. Decide on one common value
C. Avoid communication
D. Increase randomness

👉 **Answer: B**

🔹 Level 6: Properties of Consensus ⚠️ (VERY IMPORTANT)

Q17. Termination means:
A. Nodes stop working
B. Every correct process eventually decides
C. No decision taken
D. Random execution

👉 **Answer: B**

Q18. Agreement means:
A. Nodes decide different values
B. All nodes decide same value
C. No node decides
D. Random decision

👉 **Answer: B**

Q19. Integrity means:
A. Any value can be chosen
B. Chosen value must be valid/proposed
C. No value required
D. Random value allowed

👉 **Answer: B**

🔹 Level 7: Deep Conceptual

Q20. Consensus ensures:
A. Random execution
B. Consistency across nodes
C. Faster mining
D. Less communication

👉 **Answer: B**

Q21. Which property prevents conflicting decisions?
A. Termination
B. Agreement
C. Integrity
D. Fault tolerance

👉 **Answer: B**

Q22. Which property ensures decision happens eventually?
A. Agreement
B. Integrity
C. Termination
D. Consistency

👉 **Answer: C**

🔹 Level 8: Algorithm Classification

Q23. Paxos and Raft handle:
A. Byzantine faults
B. Crash/network faults
C. No faults
D. Storage faults

👉 **Answer: B**

Q24. PBFT handles:
A. Crash faults only
B. Byzantine faults
C. No faults
D. Network faults only

👉 **Answer: B**

🔹 Level 9: Tricky Questions ⚠️

Q25. Which is TRUE?
A. Consensus is optional in distributed systems
B. Consensus ensures same decision
C. Consensus removes faults
D. Consensus avoids communication

👉 **Answer: B**

Q26. Consensus algorithms must handle:
A. Only correct nodes
B. Failures in system
C. No communication
D. Only storage

👉 **Answer: B**

Q27. If agreement fails:
A. All nodes same state
B. Nodes may diverge
C. Faster execution
D. No issue

👉 **Answer: B**

🔥 Ultra-Tricky (Exam Killer Round)

Q28. Consensus is hardest to achieve under:
A. Single node
B. Faulty distributed system
C. No communication
D. Static system

👉 **Answer: B**

Q29. Which combination defines correct consensus?
A. Random + Fast

B. Termination + Agreement + Integrity
C. Storage + Speed
D. Execution + Mining

👉 **Answer: B**

Q30. Final purpose of consensus is:
A. Increase storage
B. Ensure single agreed value across nodes
C. Reduce nodes
D. Increase randomness

👉 **Answer: B**

✅ You've mastered

- Definition ✔️
- Need ✔️
- Applications ✔️
- Fault types ✔️
- Formal problem ✔️
- Properties (VERY IMPORTANT) ✔️
- Algorithm categories ✔️

🚀 Next step

Now we go deeper into **actual algorithms (very exam-heavy)**

👉 Say **"next topic"** → we start **Paxos Algorithm (most tricky + conceptual)**
OR
👉 **"test me"** → full mixed MCQ exam from ALL topics so far (high difficulty)

next topic

Brace yourself—**Paxos is where exams try to trick you**. Focus on roles, phases, and edge cases.

🎯 TOPIC 8: PAXOS ALGORITHM — MCQ SET

🔹 Level 1: Core Basics

Q1. Paxos is designed to solve:
A. Storage problem
B. Distributed consensus problem
C. Encryption problem
D. Networking issue

👉 **Answer: B**

Q2. Paxos was proposed by:
A. Satoshi Nakamoto
B. Leslie Lamport
C. Vitalik Buterin
D. Turing

👉 **Answer: B**

Q3. Main goal of Paxos:
A. Multiple values selection
B. Choose a single agreed value
C. Remove nodes
D. Avoid communication

👉 **Answer: B**

🔹 Level 2: Roles in Paxos ⚠️

Q4. Which is NOT a Paxos role?
A. Proposer
B. Acceptor
C. Learner
D. Miner

👉 **Answer: D**

Q5. Who proposes a value?
A. Acceptor
B. Learner
C. Proposer
D. Network

👉 **Answer: C**

Q6. Who decides whether to accept a proposal?
A. Proposer
B. Acceptor

C. Learner
D. Client

👉 **Answer: B**

Q7. Who learns the final value?
A. Proposer
B. Acceptor
C. Learner
D. Leader

👉 **Answer: C**

🔹 Level 3: Proposal Concept

Q8. A proposal in Paxos consists of:
A. Value only
B. Number only
C. Number + value
D. Node ID only

👉 **Answer: C**

Q9. Proposal number is used to:
A. Store data
B. Order proposals
C. Encrypt messages
D. Reduce nodes

👉 **Answer: B**

🔹 Level 4: Phases of Paxos ⚠️

Q10. First phase of Paxos:
A. Accept
B. Learn
C. Prepare
D. Commit

👉 **Answer: C**

Q11. Second phase of Paxos:
A. Learn
B. Accept
C. Prepare
D. Execute

👉 **Answer: B**

Q12. Final phase is:
A. Commit
B. Learn
C. Prepare
D. Propose

👉 **Answer: B**

🔹 Level 5: Prepare Phase

Q13. In prepare phase, proposer sends:
A. Accept request
B. Prepare request
C. Commit message
D. Learn message

👉 **Answer: B**

Q14. Acceptor responds if:
A. Proposal number is smaller
B. Proposal number is higher
C. No proposal exists
D. Randomly

👉 **Answer: B**

🔹 Level 6: Accept Phase

Q15. In accept phase, proposer sends:
A. Prepare request
B. Accept request
C. Learn message
D. Abort message

👉 **Answer: B**

Q16. Acceptors accept proposal if:
A. It is random
B. It meets promise conditions
C. It is smallest
D. It is ignored

👉 **Answer: B**

🔹 Level 7: Majority Concept ⚠️

Q17. Paxos requires agreement from:
A. All nodes
B. Majority of acceptors
C. One node
D. No nodes

👉 **Answer: B**

Q18. Majority means:
A. N/2
B. > N/2
C. N
D. < N/2

👉 **Answer: B**

🔹 Level 8: Failure Handling

Q19. Paxos can tolerate:
A. No failures
B. Crash failures
C. Byzantine failures
D. Only network failures

👉 **Answer: B**

Q20. If majority of acceptors fail:
A. Consensus still works
B. Consensus fails
C. Faster decision
D. No issue

👉 **Answer: B**

🔹 Level 9: Proposer Failure

Q21. If proposer fails:
A. System stops permanently
B. Another proposer can continue
C. Consensus ends
D. No value chosen

👉 **Answer: B**

🔹 Level 10: Dueling Proposers ⚠️

Q22. Dueling proposers problem occurs when:
A. No proposer exists
B. Multiple proposers compete
C. Only one proposer exists
D. No acceptor exists

👉 **Answer: B**

Q23. Conflict is resolved using:
A. Random choice
B. Higher proposal number
C. Smaller proposal number
D. No rule

👉 **Answer: B**

🔹 Level 11: Leader Election

Q24. Leader election helps to:
A. Increase conflicts
B. Reduce multiple proposers
C. Remove acceptors
D. Avoid consensus

👉 **Answer: B**

🔹 Level 12: Deep Conceptual

Q25. Paxos ensures safety by:
A. Random execution
B. Majority agreement
C. No communication
D. Single node

👉 **Answer: B**

Q26. Paxos ensures liveness when:
A. Majority nodes are active
B. All nodes fail
C. No communication
D. Random execution

👉 **Answer: A**

🔹 Level 13: Tricky Questions ⚠️

Q27. Which is TRUE?
A. Paxos needs all nodes to agree
B. Paxos works with majority agreement
C. Paxos ignores failures
D. Paxos uses mining

👉 **Answer: B**

Q28. Paxos fails if:
A. One node fails
B. Majority nodes fail
C. No proposer exists
D. One acceptor fails

👉 **Answer: B**

🔥 Ultra-Tricky (Exam Killer Round)

Q29. The most critical requirement in Paxos is:
A. Storage
B. Majority agreement
C. Speed
D. Network size

👉 **Answer: B**

Q30. Final outcome of Paxos is:
A. Multiple values chosen
B. Single agreed value
C. Random value
D. No value

👉 **Answer: B**

✅ You've mastered

- Roles ✔️
- Proposal structure ✔️
- Phases ✔️
- Majority logic ✔️
- Failure handling ✔️
- Dueling proposers ✔️

- Leader election ✔️

🚀 Next step

Next is **Raft (simpler but very exam-friendly)**

👉 Say **"next topic"**
OR
👉 **"test me"** → I'll give mixed hard MCQs from ALL topics so far (real exam difficulty)

next

Nice—**Raft is your scoring zone**. Easier than Paxos, but exams love subtle traps.

🎯 TOPIC 9: RAFT CONSENSUS ALGORITHM — MCQ SET

🔹 Level 1: Core Basics

Q1. Raft is designed to:
A. Replace storage
B. Simplify distributed consensus
C. Remove nodes
D. Avoid communication

👉 **Answer: B**

Q2. Raft is an alternative to:
A. PBFT
B. Paxos
C. Blockchain
D. Mining

👉 **Answer: B**

Q3. Raft primarily handles:
A. Byzantine faults
B. Crash faults
C. No faults
D. Storage faults

👉 **Answer: B**

🔹 Level 2: Roles in Raft ⚠️

Q4. Raft has how many main roles?
A. 2
B. 3
C. 4
D. 5

👉 **Answer: B** *(Leader, Follower, Candidate)*

Q5. Which is NOT a Raft role?
A. Leader
B. Follower
C. Candidate
D. Acceptor

👉 **Answer: D**

Q6. The leader is responsible for:
A. Ignoring requests
B. Managing log replication
C. Stopping execution
D. Removing nodes

👉 **Answer: B**

🔹 Level 3: Leader-Based System

Q7. In Raft, all client requests go to:
A. Followers
B. Leader
C. Candidates
D. Random nodes

👉 **Answer: B**

Q8. Followers:
A. Act independently
B. Follow leader's instructions
C. Always become leader
D. Ignore logs

👉 **Answer: B**

🔹 Level 4: Leader Election ⚠️

Q9. Leader election occurs when:
A. System starts
B. Leader fails
C. Timeout occurs
D. Both B and C

👉 **Answer: D**

Q10. A follower becomes candidate when:
A. It receives heartbeat
B. It times out waiting for leader
C. It becomes leader directly
D. It crashes

👉 **Answer: B**

Q11. Candidate becomes leader if:
A. It gets all votes
B. It gets majority votes
C. It gets one vote
D. It waits indefinitely

👉 **Answer: B**

🔹 Level 5: Voting Process

Q12. Each node votes for:
A. Multiple candidates
B. Only one candidate per term
C. No candidate
D. Random nodes

👉 **Answer: B**

Q13. Voting decision depends on:
A. Node size
B. Term number and log index
C. Storage
D. Speed

👉 **Answer: B**

🔹 Level 6: Log Replication

Q14. Leader ensures:
A. Different logs on nodes
B. Same log entries across nodes
C. No logs
D. Random logs

👉 **Answer: B**

Q15. Log replication ensures:
A. Consistency
B. Random execution
C. Faster mining
D. Less storage

👉 **Answer: A**

Q16. Entries are committed when:
A. Leader decides alone
B. Majority of nodes replicate
C. One node accepts
D. Random nodes accept

👉 **Answer: B**

🔹 Level 7: Terms & Time ⚠️

Q17. A “term” in Raft represents:
A. Storage unit
B. Time period for leadership
C. Transaction
D. Block

👉 **Answer: B**

Q18. Term number helps in:
A. Storage
B. Ordering events and elections
C. Removing nodes
D. Mining

👉 **Answer: B**

🔹 Level 8: Fault Handling

Q19. Raft tolerates failure if:
A. All nodes fail

B. Majority nodes are active
C. Leader fails permanently
D. No nodes exist

👉 **Answer: B**

Q20. If leader fails:
A. System stops
B. New leader is elected
C. No action
D. Logs deleted

👉 **Answer: B**

🔹 Level 9: Deep Conceptual

Q21. Raft ensures consistency by:
A. Random execution
B. Leader-controlled log replication
C. No communication
D. Parallel execution

👉 **Answer: B**

Q22. Followers reject outdated leader because of:
A. Storage issues
B. Term comparison
C. Random behavior
D. No logs

👉 **Answer: B**

🔹 Level 10: Tricky Questions ⚠️

Q23. Which is TRUE?
A. Raft uses multiple leaders
B. Raft uses single leader at a time
C. Raft avoids consensus
D. Raft ignores logs

👉 **Answer: B**

Q24. Which is NOT required in Raft?
A. Leader election
B. Log replication

C. Random execution
D. Majority voting

👉 **Answer: C**

Q25. Raft fails if:
A. One node fails
B. Majority nodes fail
C. Leader changes
D. Logs replicate

👉 **Answer: B**

🔥 Ultra-Tricky (Exam Killer Round)

Q26. Raft simplifies Paxos by:
A. Removing roles
B. Using leader-based approach
C. Removing consensus
D. Ignoring failures

👉 **Answer: B**

Q27. Most critical part of Raft is:
A. Storage
B. Leader election + log replication
C. Mining
D. Encryption

👉 **Answer: B**

Q28. Which guarantees correct log order?
A. Followers
B. Leader
C. Network
D. Storage

👉 **Answer: B**

Q29. Final goal of Raft is:
A. Multiple leaders
B. Consistent replicated log
C. Faster mining
D. Random execution

👉 **Answer: B**

Q30. Raft ensures consensus by:
A. Random voting
B. Majority agreement under leader
C. Single node decision
D. No communication

👉 **Answer: B**

✅ You've mastered

- Roles ✔️
- Leader election ✔️
- Voting ✔️
- Terms ✔️
- Log replication ✔️
- Failure handling ✔️
- Paxos comparison ✔️

🚀 Next step

Now comes **Byzantine Generals Problem + BFT (VERY conceptual + tricky)**

👉 Say **"next topic"**
OR
👉 **"test me"** for full mixed exam (very high difficulty, negative marking safe)

next topic

Now we enter the **most conceptual + trick-heavy part**—this is where exams try to confuse you with scenarios.

🎯 TOPIC 10: BYZANTINE GENERALS PROBLEM & BFT — MCQ SET

🔹 Level 1: Core Basics

Q1. The Byzantine Generals Problem represents:
A. Storage issue

B. Consensus under malicious faults
C. Network speed problem
D. Encryption issue

👉 **Answer: B**

Q2. Byzantine fault means:
A. Node crashes
B. Node behaves arbitrarily or maliciously
C. Node stops communication
D. Node is slow

👉 **Answer: B**

Q3. Paxos and Raft cannot handle:
A. Crash faults
B. Network faults
C. Byzantine faults
D. Storage faults

👉 **Answer: C**

🔹 Level 2: Problem Setup ⚠️

Q4. Byzantine problem involves:
A. One node
B. Commander and lieutenants
C. Only followers
D. Only leaders

👉 **Answer: B**

Q5. The goal is for all generals to:
A. Act randomly
B. Agree on a common plan
C. Ignore messages
D. Work independently

👉 **Answer: B**

Q6. Possible decisions in classic problem:
A. Yes/No
B. Attack/Retreat
C. Start/Stop
D. Run/Wait

👉 **Answer: B**

🔹 Level 3: Fault Scenarios

Q7. A faulty commander may:
A. Send correct messages
B. Send conflicting messages to different nodes
C. Stop system
D. Execute correctly

👉 **Answer: B**

Q8. A faulty lieutenant may:
A. Follow instructions correctly
B. Ignore or alter received message
C. Not exist
D. Control all nodes

👉 **Answer: B**

🔹 Level 4: Key Issues ⚠️

Q9. Byzantine faults break:
A. Storage
B. Integrity and agreement
C. Execution speed
D. Block size

👉 **Answer: B**

Q10. Majority voting fails when:
A. All nodes are honest
B. Malicious nodes send conflicting information
C. No communication exists
D. Only one node exists

👉 **Answer: B**

🔹 Level 5: BFT Theory

Q11. To tolerate f faulty nodes, system needs:
A. f + 1 nodes
B. 2f nodes
C. 3f + 1 nodes
D. f nodes

👉 **Answer: C**

Q12. Minimum nodes required for 1 faulty node:
A. 2
B. 3
C. 4
D. 5

👉 **Answer: C** *(3×1 + 1 = 4)*

Q13. For f faults, number of correct nodes required is at least:
A. f
B. 2f + 1
C. 3f
D. f + 2

👉 **Answer: B**

🔹 Level 6: System Model

Q14. Byzantine model assumes:
A. Unknown sender identity
B. Known sender identity
C. No communication
D. Infinite nodes

👉 **Answer: B**

Q15. Communication is assumed to be:
A. Unreliable
B. Reliable
C. Random
D. Optional

👉 **Answer: B**

Q16. Network is assumed to be:
A. Partially connected
B. Fully connected
C. No connection
D. Random

👉 **Answer: B**

🔹 Level 7: Algorithm Insight

Q17. Lamport–Shostak–Pease algorithm works in:
A. Asynchronous system

B. Synchronous system
C. No communication system
D. Centralized system

👉 **Answer: B**

Q18. It uses:
A. Mining
B. Broadcast and relay messages
C. Storage
D. Encryption

👉 **Answer: B**

Q19. Decision is taken using:
A. Random choice
B. Majority voting after rounds
C. Single node
D. No decision

👉 **Answer: B**

🔹 Level 8: Deep Conceptual

Q20. Byzantine faults are hardest because:
A. Nodes stop
B. Nodes behave unpredictably
C. No communication
D. No execution

👉 **Answer: B**

Q21. Which is TRUE?
A. Crash faults are harder than Byzantine
B. Byzantine faults include malicious behavior
C. Byzantine faults are easy
D. No faults exist

👉 **Answer: B**

🔹 Level 9: Tricky Questions ⚠️

Q22. Which is NOT a Byzantine behavior?
A. Sending conflicting messages
B. Lying about state

C. Node crash
D. Arbitrary behavior

👉 **Answer: C**

Q23. Why is 3f+1 needed?
A. For storage
B. To outvote faulty nodes
C. To reduce nodes
D. To increase speed

👉 **Answer: B**

Q24. If nodes < 3f+1:
A. Consensus always works
B. Byzantine agreement may fail
C. Faster execution
D. No issue

👉 **Answer: B**

🔥 Ultra-Tricky (Exam Killer Round)

Q25. With 1 faulty node and 3 total nodes:
A. Consensus works
B. Consensus fails
C. Faster agreement
D. No communication

👉 **Answer: B** *(Need at least 4 nodes)*

Q26. Byzantine fault tolerance ensures:
A. No faults
B. Correct consensus despite malicious nodes
C. Faster execution
D. No communication

👉 **Answer: B**

Q27. Main challenge in Byzantine systems:
A. Storage
B. Trust and verification
C. Speed
D. Memory

👉 **Answer: B**

Q28. Which system is hardest to design?
A. Single node
B. Crash fault system
C. Byzantine fault system
D. No node system

👉 **Answer: C**

Q29. Byzantine agreement ensures:
A. Random outputs
B. Agreement despite malicious nodes
C. No communication
D. No decision

👉 **Answer: B**

Q30. Final goal of BFT is:
A. Remove nodes
B. Achieve consensus under adversarial conditions
C. Increase storage
D. Reduce computation

👉 **Answer: B**

✅ You've mastered

- Byzantine problem ✔️
- Fault types ✔️
- 3f+1 rule ✔️
- System assumptions ✔️
- LSP algorithm ✔️
- Failure scenarios ✔️
- Exam traps ✔️

🚀 Next step

Now comes **PBFT (Practical Byzantine Fault Tolerance)**
👉 VERY IMPORTANT + many MCQs

Say:
👉 **"next topic"**
OR
👉 **"test me"** (full syllabus mixed, high difficulty, negative marking safe)

next topic

Final stretch—the **most exam-heavy algorithm**. PBFT questions often hinge on *counts, phases, and conditions*.

🎯 TOPIC 11: PBFT (Practical Byzantine Fault Tolerance) — MCQ SET

🔹 Level 1: Core Basics

Q1. PBFT is designed to handle:
A. Crash faults
B. Byzantine faults
C. Storage faults
D. No faults

👉 **Answer: B**

Q2. PBFT works in:
A. Synchronous systems only
B. Asynchronous systems
C. Single-node systems
D. Offline systems

👉 **Answer: B**

Q3. PBFT stands for:
A. Partial BFT
B. Practical Byzantine Fault Tolerance
C. Parallel BFT
D. Private BFT

👉 **Answer: B**

🔹 Level 2: Why PBFT

Q4. PBFT is called “practical” because:
A. It removes faults
B. It works efficiently in real systems

C. It avoids communication
D. It uses mining

👉 **Answer: B**

Q5. PBFT ensures:
A. Random outputs
B. Safety and liveness
C. Only speed
D. Only storage

👉 **Answer: B**

🔹 Level 3: System Model ⚠️

Q6. PBFT requires at least:
A. 2f + 1 nodes
B. 3f + 1 nodes
C. f + 1 nodes
D. 4f nodes

👉 **Answer: B**

Q7. To tolerate 1 faulty node, PBFT needs:
A. 2 nodes
B. 3 nodes
C. 4 nodes
D. 5 nodes

👉 **Answer: C**

Q8. PBFT system consists of:
A. Only leader
B. Client, primary, replicas
C. Only replicas
D. Only clients

👉 **Answer: B**

🔹 Level 4: Roles

Q9. Primary node is:
A. Backup
B. Leader
C. Client
D. Follower

👉 **Answer: B**

Q10. Replicas are:
A. Clients
B. Backup nodes
C. Leaders
D. Miners

👉 **Answer: B**

🔹 Level 5: Basic Working Flow

Q11. First step in PBFT:
A. Replica executes
B. Client sends request to primary
C. Commit phase
D. View change

👉 **Answer: B**

Q12. Primary then:
A. Executes directly
B. Multicasts request to replicas
C. Stops execution
D. Deletes request

👉 **Answer: B**

Q13. Client waits for:
A. One reply
B. f + 1 matching replies
C. All replies
D. No reply

👉 **Answer: B**

🔹 Level 6: Three Phases ⚠️ (VERY IMPORTANT)

Q14. First phase:
A. Prepare
B. Commit
C. Pre-prepare
D. Execute

👉 **Answer: C**

Q15. Second phase:
A. Commit
B. Prepare
C. Learn
D. Execute

👉 **Answer: B**

Q16. Third phase:
A. Execute
B. Commit
C. Prepare
D. Learn

👉 **Answer: B**

🔹 Level 7: Pre-Prepare Phase

Q17. Pre-prepare includes:
A. Only value
B. View number, sequence number, digest
C. Only signature
D. Only node ID

👉 **Answer: B**

Q18. Purpose of sequence number:
A. Encryption
B. Ordering requests
C. Storage
D. Speed

👉 **Answer: B**

🔹 Level 8: Prepare Phase

Q19. Replicas verify:
A. Random data
B. Signature and correctness
C. Only value
D. Only client

👉 **Answer: B**

Q20. Prepare phase ensures:
A. Random execution

B. Agreement on request
C. Faster mining
D. No communication

👉 **Answer: B**

🔹 Level 9: Commit Phase ⚠️

Q21. Commit requires at least:
A. f votes
B. f + 1 votes
C. 2f + 1 votes
D. 3f votes

👉 **Answer: C**

Q22. Replica commits when:
A. One message received
B. 2f + 1 commit messages received
C. No message
D. Random decision

👉 **Answer: B**

🔹 Level 10: Correctness

Q23. PBFT ensures:
A. Random ordering
B. Total ordering of requests
C. No ordering
D. Partial ordering

👉 **Answer: B**

Q24. Agreement means:
A. Different states
B. Same state across replicas
C. No execution
D. Random execution

👉 **Answer: B**

🔹 Level 11: View Change ⚠️

Q25. View change occurs when:
A. System works fine
B. Primary fails
C. No nodes exist
D. No communication

👉 **Answer: B**

Q26. Trigger for view change:
A. Random event
B. Timeout
C. Storage full
D. Execution complete

👉 **Answer: B**

Q27. New primary is selected to ensure:
A. System stops
B. Liveness
C. Storage
D. Random execution

👉 **Answer: B**

🔹 Level 12: Deep Concepts

Q28. Safety means:
A. System stops
B. All correct nodes agree
C. No communication
D. Random outputs

👉 **Answer: B**

Q29. Liveness means:
A. System halts
B. System continues progress
C. No nodes exist
D. No execution

👉 **Answer: B**

🔥 Ultra-Tricky (Exam Killer Round)

Q30. Why 3f+1 nodes are required in PBFT?
A. For storage

B. To tolerate f Byzantine faults
C. For speed
D. For mining

👉 **Answer: B**

Q31. Why 2f+1 commits required?
A. Random selection
B. To outvote faulty nodes
C. For storage
D. For speed

👉 **Answer: B**

Q32. If commits < 2f+1:
A. System commits
B. No consensus
C. Faster execution
D. No issue

👉 **Answer: B**

Q33. PBFT mainly improves over theoretical BFT by:
A. Removing faults
B. Working in asynchronous real systems
C. Reducing nodes
D. Removing communication

👉 **Answer: B**

✅ You've mastered EVERYTHING

- PBFT architecture ✔️
- 3 phases ✔️
- Message flow ✔️
- 3f+1 & 2f+1 logic ✔️
- View change ✔️
- Safety & liveness ✔️

🧠 FINAL STATUS

You've now covered **entire syllabus with zero gaps**:

- Permissioned ✔️
- Smart Contracts ✔️

- Limitations ✔️
- Determinism ✔️
- Execution ✔️
- SMR ✔️
- Consensus ✔️
- Paxos ✔️
- Raft ✔️
- BFT ✔️
- PBFT ✔️

🚀 FINAL STEP (VERY IMPORTANT)

Now do **real exam simulation**:

Say:
👉 **"test me full syllabus"**

I'll give:

- Mixed MCQs (hard)
- No answers (like exam)
- Negative marking safe tricks

This is where you lock **100% accuracy**.

Give me those questions from full syllabus which are highly likely to come in exam

Good—now we switch to **high-probability, exam-focused MCQs only**.
These are the ones most likely to appear (concept + trap-heavy).
No explanations, only answers—train like real exam.

🎯 FULL SYLLABUS — HIGH PROBABILITY MCQs

🔹 SECTION 1: Core Concepts (Very Likely)

Q1. Permissioned blockchain requires consensus because:
A. Nodes are anonymous

B. Nodes partially trust each other
C. No communication exists
D. No transactions exist

👉 **Answer: B**

Q2. Smart contracts execute when:
A. Block is mined
B. Conditions are satisfied
C. Network fails
D. User logs out

👉 **Answer: B**

Q3. Sequential execution leads to:
A. High throughput
B. Throughput bottleneck
C. No delay
D. Better scalability

👉 **Answer: B**

Q4. Non-deterministic execution causes:
A. Same outputs
B. Forks
C. Faster consensus
D. Reduced computation

👉 **Answer: B**

Q5. Executing smart contracts on all nodes leads to:
A. Efficiency
B. Redundant computation
C. Less security
D. Faster execution

👉 **Answer: B**

Q6. SMR requires:
A. Random execution
B. Same inputs in same order
C. No inputs
D. Different outputs

👉 **Answer: B**

Q7. Consensus ensures:
A. Different decisions

B. Same decision across nodes
C. No communication
D. Faster mining

👉 **Answer: B**

🔹 SECTION 2: Consensus Properties ⚠️ VERY IMPORTANT

Q8. Termination means:
A. Nodes stop
B. All correct nodes eventually decide
C. No decision
D. Random decision

👉 **Answer: B**

Q9. Agreement means:
A. Different values
B. Same value for all nodes
C. No execution
D. Random value

👉 **Answer: B**

Q10. Integrity ensures:
A. Any value accepted
B. Only valid proposed value chosen
C. Random value
D. No value

👉 **Answer: B**

🔹 SECTION 3: Paxos (HIGH WEIGHT)

Q11. Paxos requires:
A. All nodes agreement
B. Majority agreement
C. Single node
D. No nodes

👉 **Answer: B**

Q12. Paxos phases correct order:
A. Accept → Prepare → Learn
B. Prepare → Accept → Learn

C. Learn → Prepare → Accept
D. Prepare → Learn → Accept

👉 **Answer: B**

Q13. Proposal consists of:
A. Value only
B. Number only
C. Number + value
D. Node ID

👉 **Answer: C**

Q14. Paxos fails when:
A. One node fails
B. Majority fails
C. One proposer fails
D. One acceptor fails

👉 **Answer: B**

🔹 SECTION 4: Raft (VERY LIKELY)

Q15. Raft uses:
A. Multiple leaders
B. Single leader
C. No leader
D. Random nodes

👉 **Answer: B**

Q16. Leader is elected when:
A. Storage full
B. Timeout occurs
C. No logs
D. Network fast

👉 **Answer: B**

Q17. Log entries are committed when:
A. One node accepts
B. Majority accepts
C. All nodes accept
D. No node accepts

👉 **Answer: B**

Q18. Raft ensures consistency via:
A. Random execution
B. Leader-based log replication
C. No communication
D. Independent execution

👉 **Answer: B**

🔹 SECTION 5: Byzantine Faults ⚠️ CRITICAL

Q19. Byzantine fault means:
A. Node crash
B. Malicious behavior
C. Slow node
D. No communication

👉 **Answer: B**

Q20. Paxos/Raft cannot handle:
A. Crash faults
B. Byzantine faults
C. Network faults
D. Storage faults

👉 **Answer: B**

Q21. Minimum nodes required to tolerate f faults:
A. 2f
B. 3f + 1
C. f + 1
D. 4f

👉 **Answer: B**

Q22. For 1 faulty node, required nodes:
A. 3
B. 4
C. 2
D. 5

👉 **Answer: B**

🔹 SECTION 6: PBFT ⚠️ MOST IMPORTANT

Q23. PBFT works in:
A. Synchronous only

B. Asynchronous systems
C. Single node
D. Offline system

👉 **Answer: B**

Q24. PBFT phases order:
A. Prepare → Commit → Pre-prepare
B. Pre-prepare → Prepare → Commit
C. Commit → Prepare → Pre-prepare
D. Prepare → Pre-prepare → Commit

👉 **Answer: B**

Q25. Commit requires:
A. f votes
B. f+1 votes
C. 2f+1 votes
D. 3f votes

👉 **Answer: C**

Q26. Client waits for:
A. 1 reply
B. f+1 replies
C. All replies
D. No replies

👉 **Answer: B**

Q27. View change occurs when:
A. System fast
B. Primary fails
C. No nodes
D. No execution

👉 **Answer: B**

Q28. Safety means:
A. System stops
B. All correct nodes agree
C. No communication
D. Random outputs

👉 **Answer: B**

Q29. Liveness means:
A. System halts

B. System continues progress
C. No nodes
D. No execution

👉 **Answer: B**

🔥 FINAL EXAM TRAPS (VERY HIGH PROBABILITY)

Q30. Determinism is required to:
A. Increase speed
B. Ensure same state across nodes
C. Reduce storage
D. Remove consensus

👉 **Answer: B**

Q31. Biggest limitation of permissionless systems:
A. Security
B. Sequential execution
C. Storage
D. Network

👉 **Answer: B**

Q32. SMR fails if:
A. Same input order
B. Non-deterministic execution
C. Determinism exists
D. Replication exists

👉 **Answer: B**

Q33. Consensus is hardest under:
A. Single node
B. Byzantine faults
C. No communication
D. Static system

👉 **Answer: B**

🧠 FINAL ADVICE (READ THIS CAREFULLY)

In exam, **avoid losing marks**:

- Eliminate options with:
    - ❌ “random”

    - ❌ “no consensus needed”
    - ❌ “all nodes always required”
- Focus on:
    - ✔️ Majority logic
    - ✔️ Determinism
    - ✔️ Same state

🚀 If you want guaranteed full marks:

Say:
👉 **“give me hardest tricky MCQs”** (last-level prep)
OR
👉 **“rapid revision cheatsheet”** (1-page memory before exam)